Analiza Algoritmilor

Seria CA
Anul 2013 – 2014

Temă de casă

Versiuni:
v1 – 17/11/2013 – Publicarea primei versiuni a temei de casa

Problema 1 – 16.5p

a) Fie A, B, C multimi recursiv-numarabile. Ce puteti spune despre multimile urmatoare?

$D = \{ x \mid \forall y , \exists z, (x, y, z) \notin A\}$

$E = \{ x \mid \exists y, \exists z, (x, y, z) \in A \wedge (x, z) \in B \wedge (x, y) \in \mathcal{B} \wedge (x, y) \in \mathcal{C}\}$

b) Fie B o submultime din multime numerelor naturale. In functie de tipul multimii B (recursiva, recursiv enumarabila, nerecursiva) ce puteti spune despre complementul lui B?

c) Fie predicatele P1, P2, Q1, Q2 : R -> {0,1} . Stim ca predicatele P1 v P2, Q1 v Q2 sunt semidecidabile, iar P2 = ~Q2. Ce puteti spune despre predicatul P1 v Q1?

Observatie! Trebuie sa justificati raspunsurile oferite pentru toate cele 3 puncte.

Problema 2 – 6.5p

Ce puteti spune despre decidabilitatea problemei urmatoare: Dandu-se un program oarecare P: N $\rightarrow$ N, se termina P(5)?

Problema 3 - 8p

Avem 4 algoritmi despre care stim urmatoarele afirmatii: algoritmul A are complexitate O(n), algoritmul B are complexitate $o(n^2 \log n)$, algoritmul C are complexitate $\omega(n \log\log n)$, iar algoritmul D are o complexitate intre $\Omega(n^2 \log^2 n)$ si $O(n^3)$. Ce relatii de ordine putem stabili intre acesti algoritmi din punct de vedere al complexitatii? Justificati.

Problema 4 – 16p

Pentru fiecare afirmatie de mai jos, scrieti valoarea de adevar (mereu adevarat, uneori adevarat, mereu fals) impreuna cu demonstratia necesara.

a) Daca g = O(f) si h = O(f) atunci g = O(h)

b)Daca $f(n) = \Omega(3^n)$ si g(n) = O(n) atunci $f(n) = \Omega(3^{g(n)})$

c) Daca $f(n) = \Omega(n^5)$ si $g(n) = O(\sqrt{n})$ atunci $f(g(n)) = \omega(n^2)$

d) Daca f(n) = O(g(n)) si $f(n) \neq o(g(n))$ atunci $f(n) = \Theta(g(n))$

e) Daca $f(n)g(n) = \Omega(h(n)^2)$ atunci h(n) = O(f(n) + g(n))

Problema 5 - 8p

Se da urmatoarea problema: O bibliotecara are asignat pentru fiecare carte un numar care reprezinta de cate ori a fost imprumutata. Cartile se afla intr-o ordine aleatoare. Bibliotecara doreste sa extraga cartea care se situeaza pe pozitia k in top-ul celor mai imprumutate carti.

a) Scrieti un algoritm nerecursiv, cat mai eficient, care rezolva aceasta problema si determinati complexitatea sa.

b) Scrieti un algoritm recursiv, cat mai eficient, care rezolva aceasta problema si determinati complexitatea sa.

Problema 6 - 18p

a) Gasiti o solutie pentru urmatoarea recurenta prin una dintre metodele prezentate la curs sau laborator si apoi demonstrati prin metoda substitutiei ca solutia aceasta este corecta:

$T(n) = T(n/2 + 2\text{sqrt}(n)) + n$

b) Sa se verifice daca se poate aplica teorema master pentru recurentele. In cazul in care se poate aplica, calculati ordinul de crestere al fiecarei recurente folosing teorema master, altfel gasiti o solutie alternativa pentru rezolvarea recurentei:

b1) $T(n) = 6T(n/2) + 2^{3 \log_2 n}$

b2) $T(n) = 8T(n/2) + n^3/(\log_2 n)^4$

b3) $T(n) = 9T(n/3) + n(\log_2 n)^3$

Problema 7 - 8p

Determinati relatia de recurenta pentru urmatorul algoritm mai eficient de inmultire a 2 numere intregi, explicand ce implica pasii de divide, impera si combina si complexitatea fiecarui pas. Apoi calculati complexitatea intregului algoritm printr-o metoda la alegere. Justificati de ce este acest algoritm mai bun decat algoritmul clasic de inmultire a doua numere intregi.

```
// vrem sa inmultim doua numere mari in baza 10, a si b, fiecare avand maxim n cifre
better-multiply (a[1..n], b[1..n]) {
       identify a1[1..n/2], a2[1..n/2] such that a = a1 + a2 * 10^(n/2)
       identify b1[1..n/2], b2[1..n/2] such that b = b1 + b2 * 10^(n/2)

       // inmultind a * b =
       // = (a1 + a2 * 10^(n/2)) * (b1 + b2 * 10^(n/2))
       // = a1 * b1 + (a2 * b1 + a1 * b2)  * 10^(n/2) + a2 * b2 * 10^n
       // in expresia de mai sus, avem 4 produse de numere, deci nu ne convine acest
       // aspect; vrem sa il reducem
       // notam q1 = (a1 * b1)
       // q3 = a2 * b2;
       // si q2 = (a1 + a2) * (b1 + b2) = (a1 * b2 + a2 * b1) + q1 + q3
       // deci a * b =  q1 + (q2 - q1 - q3)  * 10^(n/2) + q3 * 10^n
       // Observam ca in aceasta situatie trebuie sa calculam doar
       // 3 produse de numere q1, q2 si q3. In plus avem niste adunari care
       // sunt mai "ieftine", iar inmultirile si ridicarile la putere
       // care mai apar in formula sunt considerate si ele "ieftine"
       // (De ce? pentru ca in baza 2, se realizeaza cu shiftare pe biti)
       q1 = better-multiply(a1, b1)
       q2 = better-multiply(a1 + a2, b1 + b2)
       q3 = better-multiply(a2, b2)

       RETURN q1 + (q2 - q1 - q3) * 10^(n/2) + q3 * 10^n
}
```

Problema 8 - 9p

Sa consideram structura de data urmatoare: un contor *i* (adica un numar intreg) de la *1* la *n* care are definite doua operatii: *INC(i, n)* si *DEC(i, n)*. Initial, contorul este pe pozitia *i=n/4*, iar operatiile INC si DEC sunt definite astfel:

`INC(i, n)`
`  IF (i<n)`
`    i++`
	`DEC(i, n)`
	`  IF (i>1)`
	`    i--`

ELSE WHILE (i>n/4) i--	ELSE WHILE (i<n/4) i++

Calculati costurile in cazul cel mai defavorabil pentru cele 2 operatii fara a folosi analiza amortizata. Apoi, definiti o functie potential pentru acest contor si demonstrati ca exista un cost amortizat constant pentru INC si DEC, indiferent de momentul in care acestea se executa.

Problema 9 - 13p

Se considera o stiva implementata folosind un vector. Operatiile definite pentru stiva sunt *push* si *pop*. Totusi, cand se adauga un element in stiva (push) si vectorul este plin, trebuie sa se aloce un nou vector de dimensiune mai mare si sa se copieze elementele din vectorul vechi in cel nou, iar apoi se va folosi vectorul nou pe post de stiva.

a) Care este complexitatea operatiilor push si pop in cazul cel mai defavorabil? Justificati.

b) Daca dimensiunea vectorului se incrementeaza cu 1 atunci cand este necesar un vector mai mare, care este costul total pentru n operatii si care este costul mediu (amortizat) al unei operatii pe stiva, folosind metoda agregarii ?

c) Daca dimensiunea vectorului se tripleaza atunci cand este necesar un vector mai mare, care este costul total pentru n operatii si care este costul mediu (amortizat) al unei operatii pe stiva, folosind metoda agregarii ?

d) Gasiti o functie potential pentru a demonstra ca se poate obtine un cost amortizat constant pentru cele doua operatii, in cazul triplarii vectorului folosita la punctul anterior.

Problema 10 - 10p

Fie urmatoarele doua tipuri de date abstracte:

Tree<T>, un arbore binar ce contine in nodurile interne elemente de tipul T, definit prin constructorii de baza urmatori:

empty : -> Tree<T>

Node : Tree<T> x T x Tree<T> -> Tree<T>

List<T>, o lista de elemente de tipul T, definit prin constructorii de baza urmatori si operatorul ++ definit mai jos:

[] : -> List<T>

(:) : T x List<T> -> List<T>

(++) : List<T> x List<T> -> List<T>

[] ++ l = l

(e:l) ++ l' = e: (l ++ l')

Fie P(t) urmatoarea proprietate: "pentru orice nod n din arborele t, valoarea lui n este mai mare sau egala decat orice valoare din subarborele stang al lui n, si mai mica sau egala decat orice valoare din subarborele drept al lui n"

a) Definiti axiome pentru operatia urmatoare:
flatten: Tree<T> -> List<T>

astfel incat urmatoarea proprietate sa fie adevarata:
$\forall$ t $\in$ Tree<T>, P(t) $\rightarrow$ flatten(t) este o lista sortata.

b) Demonstrati faptul ca specificatia voastra respecta proprietatea enuntata mai sus.

Problema 11 - 18p

Pentru tipul ***LIST*** definit prin constructorii de baza:
[] : -> LIST
[a] : T -> LIST
a::x (sau cons(a, x)) : T X LIST -> LIST (adica ***a*** adaugat la inceputul listei ***l***)

a) Definiti functiile:
member(a, l) – verifica apartenenta elementului ***a*** in lista ***l***
maxelem(l), minelem(l) – intorc elementul maxim, respectiv minim din lista ***l***
remove(l, a) : LIST x T -> LIST - intoarce o lista rezultata din eliminarea elementelor ***a*** din lista ***l***
triple(l, a) : LIST x T -> LIST - intoarce o lista rezultat din triplarea elementelor ***a*** din ***l*** (fiecare ***a*** devine ***aaa***)
reverse(l) – inverseaza elementele din lista ***l***
count(l, a) – numara aparitiile elementului ***a*** in lista ***l***
size(l) – intoarce numarul de elemente din lista ***l***

Folosind inductia structurala, demonstrati urmatoarele proprietati:
b) 3 * count(l, a) + size(remove(l, a)) == size(triple(l, a))
c) member(a, l) $\rightarrow$ (minelem(l) <= a <= maxelem(l)) AND (member(a,reverse(l)))

Problema 12 - 11p

Fie algoritmul urmator care gaseste subsecventa de suma minima dintr-un vector:

```
MinSum (a[], n){
       k = 1;
       t = a[0];
       s = a[0];
       while (k != n) {
              t = min(t + a[k], a[k]);
              s = min(s,t);
              k = k + 1;
       }
}
```

si asertiunile:
A1: $\forall i, j\ (0 \le i \le j < n \rightarrow s \le S_{i,j})$
A2: $\exists i, j\ (0 \le i \le j < n \wedge s = S_{i,j})$

a) Explicati in propriile voastre cuvinte semnificatia celor doua asertiuni A1 si A2, stiind ca $S_{i,j} = \sum_{k=i}^{j} a[k]$.
b) Demonstrati corectitudinea algorimului MinSum folosind invarianti la ciclare. Puteti sa incercati sa folositi cele doua proprietati definite mai sus (demonstrand astfel si corectitudinea acestora).

Problema 13 - 17p

a) Scrieţi un algoritm nedeterminist pentru problema: *se da un graf G = (V, E), astfel incat fiecare nod are asociata o pondere intreaga, si un intreg X. Se poate gasi un subset de noduri din V astfel incat intre oricare doua noduri din subset nu exista muchie si suma ponderilor nodurilor din subset sa fie mai mare sau egala decat X?*

b) Scrieţi un algoritm nedeterminist pentru problema: *dandu-se doua grafuri G1 si G2 sa se verifice daca G1 este izomorf cu un subgraf al lui G2.*

Determinati si justificati complexitatea temporala si spatiala pentru ambii algoritmi nedeterministi construiti.

Problema 14 - 11p

Demonstrati ca problema AcoperirePlana de mai jos este NP-completa.

*Problema AcoperirePlana: Se dau m coordonate reale pe axa Ox: x_1, x_2, .., x_m si 3 coordonate reale pe axa Oy: y_1, y_2, y_3. Din acestea se formeaza setul S de 3*m puncte in plan cu coordonatele (x_i, y_j), i=1..m, j=1..3. Pornind de la setul S se contruieste o multime M ce contine tupluri de cate 3 puncte din S (nu este obligatoriu ca multimea M sa contina toate tuplurile de cate 3 puncte ce se pot defini peste S). Exista o submultime M'⊆M astfel incat fiecare punct din S sa apara exact intr-un tuplu din M' ?*

Problema 15 - 20p

a) Demonstrati ca problema *ciclului-hamiltonian* este NP-completa. Pentru aceasta reduceti problema *3-SAT (*care se cunoaste ca este NP-completa) la problema *ciclului-hamiltonian*.

Problema ciclului-hamiltonian: Se da un graf neorientat G=(V,E). Un ciclu hamiltonian este definit ca fiind un ciclu care trece prin toate nodurile, vizitandu-le o singura data. Contine graful G un ciclu hamiltonian?

b) Demonstrati ca problema *ciclului-hamiltionian* se reduce polinomial la problema *comis-voiajorului*.

*Problema comis-voiajorului: Un comis voiajor trebuie sa viziteze n orase, adica sa faca un tur vizitand fiecare oras o singura data si teminand cu orasul din care a pornit. Modeland problema pe un graf neorientat complet G=(V,E) cu n noduri si considerand un numar intreg k si o functie de cost c: V*V->Z, contine graful G un ciclu pentru comisul voiajor de cost cel mult k?*